SPN: Stable Prototypical Network for Few-Shot Learning-Based Hyperspectral Image Classification

Debabrata Pal, Valay Bundele, Biplab Banerjee, *Member, IEEE*, and Yogananda Jeppu, *Member, IEEE*

***Abstract*— We tackle the problem of few-shot image classification in the context of remote sensing hyperspectral images (HSIs). Due to the difficulties in collecting a large number of labeled training samples, the few-shot classification techniques hold much prominence in remote sensing in general. One of the bottlenecks in designing few-shot learning (FSL) systems arises from the fact that the model is likely to overfit in the presence of few training samples and the complex spectral feature distributions of the land-cover classes. To this end, we introduce a stable prototypical network (SPN) for FSL by judiciously incorporating dropout and DropBlock-based regularizers jointly within the framework and averaging model parameters using the Monte Carlo approximation. Besides, a novel variance loss term to reduce the uncertainty of the network is considered together with the cross-entropy-based classification loss to train the model in an end-to-end manner. The experimental analysis on three benchmark HSI datasets confirms the SPN's superior performance.**

***Index Terms*— DropBlock, few-shot learning (FSL), hyperspectral image (HSI) classification, uncertainty.**

I. Introduction

HYPERSPECTRAL imaging (HSI) sensors capture the reflectance from the Earth's surface in very narrow spectral bands within a wide range of the continuous electromagnetic spectra. These images are frequently utilized in a range of applications, such as carbon quantity estimation, water quality survey, coastal zones monitoring, and agriculture. HSI classification methods assign the pixels into one of the available semantic classes, primarily leveraging the useful spectral–spatial feature representations [1], [2]. Nonetheless, HSI classification is quite challenging since there is undesired scattering from neighboring pixels of different classes during image acquisition leading to the generation of *mixed pixels* in addition to the effects due to the atmospheric distortions and misalignment. Besides, HSI data also suffer from the notion of *curse of dimensionality* [3] since the number of training samples required to train high-dimensional spectral bands is usually limited causing model overfitting. To mitigate the issue, band selection [4] and dimensionality reduction (DR) were introduced to only keep important discriminative features, discarding the rest. The *ad hoc* feature engineering-based approaches [5]–[8] in this regard are subsequently replaced by the deep learning models, due to the hierarchical feature learning capabilities of the data-driven convolutional neural networks (CNNs) [9]–[11].

In addition to the success of supervised deep learning techniques in this regard [12], [13], several contemporary endeavors also consider the problem due to the availability of limited training data, for example, self-supervised feature learning using stacked autoencoders (SAEs) [3] and unsupervised feature learning using deep belief networks (DBNs) [14]. However, the performance of such models cannot match that of the supervised approaches, advocating the necessity of the presence of labeled data in designing robust HSI classification models. From a different point of view, the deep learning models are tagged as data hungry to compensate for their huge parameter space. However, as we have mentioned, obtaining training samples for HSI is nontrivial. As a remedy, alternative ways have been explored to obtain the tradeoff between the shortage of labels and the complexity of the deep learning models for HSI classification. One of the popular solutions in this respect is the few-shot learning (FSL) [15], which refers to the learning of a CNN-based classification model from very few training samples and exploiting the success of related problems.

Recently, meta-learning techniques [16], [17] have become quite popular in the FSL community. According to them, the model should not only learn transferable knowledge from a source dataset but also strengthen its ability to learn. Metric learning, which is based on meta-learning, aims to learn an efficient distance metric for classification. Siamese network, proposed in [15], has shared weights to extract features from a pair of images simultaneously. Prototypical networks learn class prototypes and perform classification using the Euclidean distance [17]. Matching networks proposed in [18] use bidirectional long short term memory (LSTM) to generate fully conditional embeddings in the support set context. Model-agnostic meta-learning (MAML) [16] seeks to meta-learn an initial configuration of weights, which can then be quickly updated for the few-shot task. Despite the importance of the FSL methods for HSI classification, there has been limited research in this field to date. In [19], HSI samples are separated in metric space using the Euclidean distance. Relation networks for HSI classification are introduced in [20] to learn a relation module that computes relation scores between the support and query embeddings [21]. However, due to high intraclass

Manuscript received March 1, 2021; revised May 5, 2021; accepted May 26, 2021. Date of publication June 15, 2021; date of current version December 28, 2021. The work of Biplab Banerjee was supported by Grant RD/0120-ISROC00-005. *(Debabrata Pal and Valay Bundele contributed equally to this work.) (Corresponding author: Debabrata Pal.)*

Debabrata Pal is with IIT Bombay, Mumbai 400076, India, and also with Honeywell Technology Solutions, Hyderabad 500032, India (e-mail: deba.iitbcsre19@gmail.com).

Valay Bundele and Biplab Banerjee are with IIT Bombay, Mumbai 400076, India (e-mail: 18d170033@iitb.ac.in; getbiplab@gmail.com).

Yogananda Jeppu is with Honeywell Technology Solutions, Bengaluru 560103, India (e-mail: yogananda.jeppu@honeywell.com).

This article has supplementary downloadable material available at https://doi.org/10.1109/LGRS.2021.3085522, provided by the authors.

Digital Object Identifier 10.1109/LGRS.2021.3085522

1558-0571 © 2021 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Shah and Anchor Kutchhi Engineering College. Downloaded on July 20,2022 at 08:18:40 UTC from IEEE Xplore. Restrictions apply.

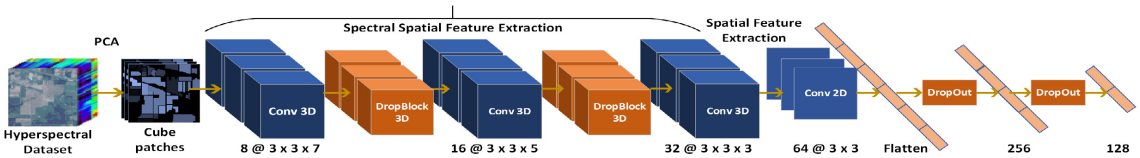

Fig. 1. Proposed hybrid spectral–spatial SPN encoder with dropout and DropBlock as regularizers for learning complex patterns of hyperspectral data.

and low interclass variance of remotely sensed data in the presence of surrounding pixel scattering, the adaptation of FSL is very hard. This causes a long convergence time with rapid flickering loss and high prediction uncertainty for the fine-grained classes.

In this letter, a stable FSL method, namely, stable prototypical networks (SPNs), is proposed, which is built upon prototypical networks [17], a centroid-based computationally fast metric-learning approach. Our approach jointly optimizes the cross-entropy (CE) loss and the true class prediction variance for the query samples. This, in effect, sharply enhances the generalization confidence of SPN. By repeatedly minimizing the prediction variance in an episode, our model fully explores the parameter space and generates stable class prototypes for better FSL. Some of the highlighting characteristics of SPN are given in the following.

1) We define an intuitive backbone network comprising 3-D, 2-D convolutions with dropout, and Dropblock as spectral–spatial regularizer for jointly exploring spectral and spatial features in FSL-based HSI classification.
2) We introduce the notion of Monte Carlo DropBlock **(MC-DropBlock)**, which minimizes the Bayesian approximation of model uncertainty.
3) Extensive experiments are performed on three benchmark HSI datasets where we find SPN to consistently outperform the related literature.
4) To the best of our knowledge, ours is among the first endeavors to use the notion of uncertainty with the FSL framework. SPN is proposed in Section II, and experiments and analysis are proposed in Section III followed by conclusion in Section IV.

II. Methodology

A. Problem Formulation

FSL is a supervised *meta-learning* approach and consists of two stages, *meta-training* and *meta-testing*. Two disjoint sets of classes, namely base classes (training set) and new classes (target set), are selected from each dataset. The target set is further divided into fine-tuning set and test set. In the HSI few-shot classification, we aim to learn a classifier $f_{\theta,\phi}$ from a small set of base classes having spectral–spatial 3-D patches per class and finally use it to classify test sample ($\hat{x}$) of unseen class, where $\phi \subset \theta$, ϕ defines feature extractor, and $(\theta - \phi)$ defines learnable few-shot classifier parameters.

During *meta-training*, parameter vector θ of f_θ is learned by sampling mini-batches or episodes. In each episode, a subset of 3-D patches from a randomly chosen set of base classes are considered as support set, $\mathcal{S} = \{(x_i, y_i)\}_{i=1}^{K}$, and another subset of remaining samples is selected as query Set, $\mathcal{Q} = \{(x_j, y_j)\}_{j=1}^{N}$, where each $x_i, x_j \in \mathbb{R}^{H\times W\times B}$ is $H \times W \times B$ dimensional feature vector. By learning features from $\mathcal{S}$ and predicting on $\mathcal{Q}$, f_θ learns transferable knowledge and also the ability to learn. If $\mathcal{S}$ contains K labeled samples for each of the C unique classes, then f_θ becomes C-way K-shot classifier.

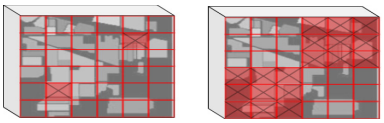

Fig. 2. (Left) Dropout randomly drops discrete neurons. (Right) Dropblock drops contiguous regions and forces other active units to learn.

A *meta-trained* f_θ can further be fine-tuned with very limited supervised samples (fine-tuning set of target classes) to quickly adapt to the unseen classification scenario. Finally, at the meta-testing phase, unlabeled test set sample $\hat{x}$ is given to f_θ to predict classes of unseen environment. This mapping function $\mathcal{S} \rightarrow f_\theta(\hat{x})$ can also be written as $P(\hat{y}|\hat{x}, \mathcal{S})$, where $\hat{y}$ refers to the unseen test class of the individual HSI dataset.

B. SPN Model Architecture

The architecture of the SPN encoder is shown in Fig. 1. Let $I \in \mathbb{R}^{H\times W\times B}$ be input spectral–spatial 3-D patch at each pixel location of HSI data, where H, W, and B represent the height, width, and the number of spectral bands, respectively. Principal component analysis (PCA) is applied to retain significant bands making patch shape as $I \in \mathbb{R}^{H\times W\times b}$, where $b < B$. Usage of 1-D-CNN and 2-D-CNN alone neglects spatial and spectral relationship, respectively. To learn latent spatial–spectral relationships jointly and increase feature maps, we use three successive 3-D-convolution layers similar to HybridSN [22]. The data cubes are then concatenated along the channel dimension. Followed by using a 2-D-convolution layer, more diverse spatial discriminative features are learned. Finally, the data are flattened, and two fully connected (FC) layers are used. Selection of hybrid 3-D, 2-D CNN architecture helps to learn hierarchical spectral–spatial correlation along with reduced model complexity.

For spatial regularization, **DropBlock** has been introduced in [23], which performs better than dropout [24] and spatial dropout [25] by considering spatial adjacency of dropping neurons. To learn spectral–spatial homogeneity from 3-D HSI patches and prevent overfitting, we use DropBlock in between Conv3D layers and Dropout in between FC layers. DropBlock applies a cubic mask of zeros at the dropout neuron positions generated by the Bernoulli distribution. The parameters of DropBlock are block size and the number of features to be dropped. Random dropping of 3-D features penalizes the model to learn structured spectral–spatial relationship of HSI data. Feature representation after applying DropBlock is shown in Fig. 2.

Authorized licensed use limited to: Shah and Anchor Kutchhi Engineering College. Downloaded on July 20,2022 at 08:18:40 UTC from IEEE Xplore. Restrictions apply.

C. Proposed SPN Training Method

We keep our base training strategy the same as that used in prototypical networks [17]. During training, an episode is sampled from the training set. Both support and query set samples are passed through the encoder to extract features. For $K > 1$, we take the mean of the generated embeddings for support samples of each class and use them as class representations. The similarity between query images and each class representation is computed using squared Euclidean distance or Bregman divergences. Using Bregman divergences [26], cluster representatives become cluster mean producing minimum distance. By applying softmax over negative of these distances, the class of query image is predicted.

Passing the episode for a single time through the encoder using stochastic gradient descent (SGD) does not extract the complete spectral–spatial distribution of HSI data in the case of prototypical networks. Dropout is less effective in regularizing convolutional layers due to the spectral–spatial feature correlation. Hence, we introduce the notion of MC-DropBlock by repeated sampling and averaging DropBlock enabled model parameters using the Monte Carlo approximation to learn complete support set distribution for uncertainty minimization. In SPN, we repeatedly pass (n_{times}) each episode through the encoder having MC-DropBlock and dropout to get multiple prediction probabilities for query samples. The confidence interval is shortened by minimizing the prediction variance for the true class to gain stability. By adding CE losses for multiple passes, we get total CE loss for an episode. The standard deviation of multiple prediction probabilities for the true class is calculated for every query sample. This is added to the total CE loss after weighting with a constant, which helps in mitigating model uncertainty. This loss is optimized for each episode. During fine-tuning, the same learning strategy is followed on the fine-tuning set.

D. Objective Function

The proposed SPN jointly optimizes loss inherited from prototypical networks [17] given in (1) along with Kullback–Leibler divergence between an approximate distribution and the posterior of deep Gaussian processes [27] by deploying DropBlock and Dropout during the meta-training stage

$$\mathcal{L}_{\text{Prototypical}} = \mathbb{E}\left[-\log \frac{\exp\left(-d\left(f_\phi(x_i), c_k\right)\right)}{\sum_{k'} \exp\left(-d\left(f_\phi(x_i), c_k\right)\right)}\right] \tag{1}$$

where $c_k \in \mathbb{R}^M$ is the class representative of class k obtained after estimating classwise mean of extracted features of support set containing K labeled samples per class. All the samples, $x_i \in \mathbb{R}^{H\times W\times b}$, are $H \times W \times b$ dimensional feature vectors. Feature extraction model f_ϕ maps $\mathbb{R}^{H\times W\times b} \to \mathbb{R}^M$, where the prototypes are M dimensional and d is the squared Euclidean distance. For each episode, prototypical networks generate a probability distribution over the classes for every query point x by calculating softmax over distances to the class representatives.

The second part of our objective function considers loss due to repeated passing of the episode through the encoder. Randomly dropping units using DropBlock and Dropout approximately estimates integration of encoder's weights or probabilistic Deep Gaussian processes [28] by marginalizing over covariance. By repeated passing, we get the average of the network's predictions, which is, in turn, Monte Carlo dropout [27].

Let C_{batch} be the set of classes for training or fine-tuning. For these classes, let the generated episodic representations be $\mathcal{E}(c) = \{e(c_i), c_i \in c_{\text{batch}}\}$. The set of query embeddings can be represented as $\mathcal{E}(Q) = \{e(q_i), q_i \in \mathcal{Q}\}$, where $\mathcal{Q}$ denotes the set of query images. Then, we calculate the Euclidean distance between each query embedding and the class representations as $\mathcal{D}_i = \{\mathcal{D}_{i1}, \mathcal{D}_{i2}, \mathcal{D}_{i3}, \ldots, \mathcal{D}_{ic}\}$ for i^{th} query image, where $\mathcal{D}_{ij} = \|e(q_i) - e(c_j)\|_2^2$ and $c_j \in c_{\text{batch}}$. The corresponding prediction probabilities for C different classes are $p_i = \{p_{i1}, p_{i2}, p_{i3}, \ldots, p_{ic}\}$, where $p_{ij} = e^{-\mathcal{D}_{ij}}/\sum_{j=1}^{c} e^{-\mathcal{D}_{ij}}$ for the ith query image and c_j^{th} class. After multiple passes (n_{times}), we get multiple prediction probabilities as $p_{i,\text{multiple}} = \{p_i^1, p_i^2, p_i^3, \ldots, p_i^{n_{\text{times}}}\}$. For the ith query image, standard deviation across all prediction probabilities is

$$\sigma_{\text{Loss},i} = \sqrt{\frac{\sum_{y=1}^{n_{\text{times}}}\left(p_{ik}^y - \mu_p\right)^2}{n_{\text{times}}}} \tag{2}$$

where $\mu_p = \sum_{y=1}^{n_{\text{times}}} p_{ik}^y / n_{\text{times}}$. Similarly, we calculate the std loss for all query images. Thus, the total std loss becomes

$$\mathcal{L}_{\text{std}} = \sum_{i=1}^{B} \sigma_{\text{Loss},i} = \sum_{i=1}^{B} \sqrt{\frac{\sum_{y=1}^{n_{\text{times}}}\left(p_{ik}^y - \mu_p\right)^2}{n_{\text{times}}}} \tag{3}$$

where B is the number of query images in an episode and k denotes the true class of ith query image. Total loss (4) is computed from (1) and (3). Here, α is a hyperparameter and $\mathcal{L}_{CE}$ is summation of $\mathcal{L}_{\text{Prototypical}}$ loss for multiple passes

$$\mathcal{L}_{\text{Total}} = \mathcal{L}_{CE} + \alpha \times \mathcal{L}_{\text{std}}. \tag{4}$$

E. Inference Strategy

During inference, the fine-tuning set is taken as support set, and the remaining samples for the test classes are taken as query set. The episodes are passed multiple times through the encoder. In each pass, feature vectors are extracted for the samples, and prediction probabilities are calculated for every class. The mean of these multiple prediction probabilities is used for the final classification.

III. Experiments and Results

A. Dataset Description and Preprocessing

We have validated the proposed SPN with three popular HSI datasets.[1] The first one, **Indian Pines (IPs)**, was captured using airborne visible/infrared imaging spectrometer (AVIRIS) sensor in northwestern Indiana with 20-m spatial resolution and 16 classes. It has 145 × 145 pixels and 220 spectral bands after removing bands covering the water absorption region. The second dataset was captured by the AVIRIS sensor at **Salinas** Valley, California, having 3.7-m spatial resolution and 16 classes. The Salinas dataset has 512 times 217 pixels and

[1] http://www.ehu.eus/ccwintco/index.php/Hyperspectral_Remote_Sensing_Scenes

Authorized licensed use limited to: Shah and Anchor Kutchhi Engineering College. Downloaded on July 20,2022 at 08:18:40 UTC from IEEE Xplore. Restrictions apply.

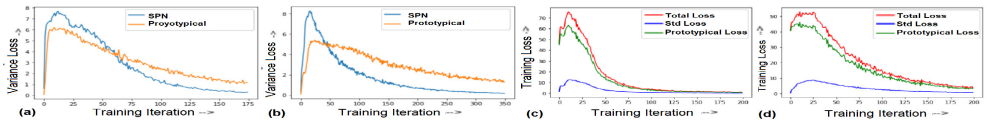

Fig. 3. SPN shows smooth exponential decay of variance loss compared to prototypical networks for (a) IPs and (b) Houston dataset. SPN optimizes both the variance loss and the prototypical (CE) loss as shown for (c) Salinas and (d) Houston datasets.

TABLE I

EFFECT OF DROPOUT AND DROPBLOCK REGULARIZERS ON FIVE-SHOT CLASSIFICATION RESULTS ON DIFFERENT DATASETS

Model	Indian Pines			Salinas			Houston		
	OA	KA	AA	OA	KA	AA	OA	KA	AA
SPN (No Drop)	84.30±0.45	79.31 ± 0.67	91.94 ± 0.26	87.26±0.27	81.33 ± 0.32	74.61 ± 0.47	68.63±0.39	60.99 ± 0.33	76.92 ± 0.17
SPN (Only Dropout)	96.32±0.31	94.99 ± 0.37	98.17 ± 0.13	94.43±0.18	92.13 ± 0.23	94.03 ± 0.51	73.00±0.41	66.52 ± 0.17	**80.57 ± 0.37**
SPN (Dropout+DropBlock)	**99.49±0.19**	**99.29 ± 0.12**	**99.75 ± 0.11**	**97.95±0.34**	**97.09 ± 0.33**	**95.18 ± 0.21**	**76.33±0.41**	**70.50 ± 0.17**	77.87 ± 0.37

TABLE II

EFFECT OF VARYING n_{TIMES} ON FIVE-SHOT CLASSIFICATION RESULTS ON DIFFERENT DATASETS

n_{times}	Indian Pines			Salinas			Houston		
	OA	KA	AA	OA	KA	AA	OA	KA	AA
1	95.33±0.23	93.55 ± 0.19	97.67 ± 0.29	86.82±0.42	82.19 ± 0.31	91.52 ± 0.41	67.78±0.39	60.49 ± 0.72	71.41 ± 0.17
5	98.16±0.11	97.43 ± 0.27	99.08 ± 0.14	91.17±0.24	87.53 ± 0.15	86.53 ± 0.61	71.92±0.43	65.13 ± 0.47	77.63 ± 0.56
10	98.33±0.17	97.66 ± 0.12	99.18 ± 0.12	**98.19±0.36**	**97.38 ± 0.51**	**98.40 ± 0.16**	70.64±0.44	63.36 ± 0.34	70.75 ± 0.43
15	98.49±0.13	97.16 ± 0.49	99.27 ± 0.12	96.55±0.21	95.13 ± 0.29	97.01 ± 0.19	76.69±0.22	71.03 ± 0.34	82.41 ± 0.11
20	99.16±0.09	98.82 ± 0.24	99.61 ± 0.05	97.03±0.13	95.77 ± 0.14	95.54 ± 0.12	**78.59±0.44**	**73.38 ± 0.34**	**84.41 ± 0.47**
25	**99.49±0.19**	**99.29 ± 0.12**	**99.75 ± 0.11**	97.95±0.34	97.09 ± 0.33	95.18 ± 0.21	76.33±0.41	70.50 ± 0.17	77.87 ± 0.37

TABLE III

COMPARISON OF CLASSIFICATION PERFORMANCE ON IPS, SALINAS, AND HOUSTON-2013 DATASETS OF THE PROPOSED SPN AND SOTA METHODS FOR $n_{\text{TIMES}} = 25$ AS SHOWN IN THE UPPER PART. THE LOWER PART SHOWS SPN'S ACCURACY IMPROVEMENT WITH SUPPORT SET SIZE

Model	Indian Pines			Salinas			Houston		
	OA	KA	AA	OA	KA	AA	OA	KA	AA
Prototypical 5-shot [23]	98.49±0.12	97.89 ± 0.23	99.39 ± 0.11	93.67±0.41	90.98 ± 0.39	88.14 ± 0.54	74.26±0.37	68.05 ± 0.46	77.12 ± 0.22
Relation 5-shot [25]	88.31±0.26	84.02 ± 0.64	85.08 ± 0.29	87.22±0.35	81.92 ± 0.41	78.75 ± 0.46	71.12±0.42	64.01 ± 0.72	69.01 ± 0.34
Matching 5-shot [28]	97.66±0.23	96.73 ± 0.13	98.56 ± 0.30	82.42±0.32	73.78 ± 0.24	68.88 ± 0.21	60.37±0.39	50.78 ± 0.42	70.50 ± 0.51
MAML 5-shot [7]	98.33±0.27	96.21 ± 0.45	97.57 ± 0.18	85.10±0.11	74.07 ± 0.17	80.24 ± 0.54	63.90±0.41	46.04 ± 0.53	**79.23 ± 0.26**
D-Res-3D-CNN 5-shot [17]	93.15±0.31	90.56 ± 0.28	92.28 ± 0.23	73.63±0.43	61.60 ± 0.46	70.82 ± 0.67	68.84±0.39	61.23 ± 0.51	73.57 ± 0.32
3D-CNN (Supervised) [17]	81.46±0.61	74.72 ± 0.47	88.26 ± 0.56	66.44±0.43	46.73 ± 0.35	55.92 ± 0.39	41.59±0.57	30.94 ± 0.45	42.10 ± 0.63
SPN 5-shot (ours)	**99.49±0.19**	**99.29 ± 0.12**	**99.75 ± 0.11**	**97.95±0.34**	**97.09 ± 0.33**	**95.18 ± 0.21**	**76.33±0.41**	**70.50 ± 0.17**	77.87 ± 0.37
SPN 1-shot (ours)	75.60±0.49	68.79 ± 0.52	80.86 ± 0.14	70.69±0.32	60.55 ± 0.65	69.03 ± 0.52	58.27±0.44	48.16 ± 0.29	65.86 ± 0.34
SPN 10-shot (ours)	99.29±0.14	98.99 ± 0.19	99.66 ± 0.07	96.59±0.29	95.15 ± 0.31	93.91 ± 0.21	79.68±0.17	74.71 ± 0.38	88.03 ± 0.29
SPN 15-shot (ours)	99.63±0.08	99.46 ± 0.14	99.83 ± 0.02	94.85±0.21	92.71 ± 0.28	90.88 ± 0.29	78.28±0.12	72.99 ± 0.19	83.31 ± 0.17

204 spectral bands after removing bands suffered by noise and water absorption. The third dataset, **Houston-2013**, which was captured at the University of Houston has 349 times 1905 pixels with 144 bands and 15 classes.

First, PCA is applied to all the datasets to reduce spectral redundancy. We choose 30 principal components for IPs and Salinas, whereas 10 are chosen for Houston-2013 keeping more than 99% spectral–spatial data variance. Then, we form cubic patches with dimensions (11,11,c) for each spatial location, using zero paddings wherever necessary. We use the standard train–test split for the classes.

B. Experimental Protocol

We took six classes for testing and the remaining for training per dataset. We considered $C = 5$, $K = 5$, and $N = 15$ during training. The Adam optimizer with learning rate 1e−5 was used. Experimentally, $\alpha = 5$ gives the best performance using cross validation. We varied hyperparameter n_{times} as {5,10,15,20,25}. For DropBlock, block size = 3 and keep probability = 0.7 were used. Five shots refer to the case with support size as 5 during testing.

C. Investigation on SPN

Fig. 3(a) and (b) shows the comparison of variance loss decay for SPN against the prototypical networks with HybridSN [22] encoder. SPN shows faster, exponential convergence of variance loss compared to linear decay in prototypical networks. Even though variance loss is initially higher in SPN due to the usage of DropBlock, it finally attains a much smaller value, which shows higher model confidence at prediction time. This is because SPN's total loss considers variance loss in addition to CE loss, as is evident from Fig. 3(c) and (d). Over each episode, randomness added due to dropout and DropBlock helps to extract a robust spectral–spatial relationship for each class and thereby learn stable class prototypes.

We estimated overall accuracy (OA), kappa accuracy (KA), and average accuracy (AA). OA defines correctly classified pixels per total sample pixels. KA shows how similar generated classification map is to the ground-truth map. AA is the average of the classwise accuracies.

The addition of DropBlock helps in extracting spatial–spectral correlation in a neighborhood rather than using only dropout. Table I shows that usage of DropBlock and dropout boosts OA by 3.17% for IP, 3.51% for Salinas, and 3.33% for Houston compared to when using only dropout. Using DropBlock along with dropout increases training time to 1.5× compared to only dropout or without any drop mechanism. We also estimated episodic standard deviation in the true class predictions by averaging over 1200 randomly sampled episodes from the test set. The episodic standard deviation has been reduced by 15% due to variance loss optimization.

Initially, as we increase n_{times}, the model is forced to extract all support set features, which helps to learn stable class representations. However, after a certain point, the increment starts hurting the learning due to the dropping of significant features. For $n_{\text{times}} = 1$, the proposed loss reduces to only CE loss leading to poor performance, as shown in Table II. At $n_{\text{times}} = 25$, the accuracy is highest for IP. For Salinas, OA increases by 7% when n_{times} is changed from 5 to 10. Table III shows that SPN's accuracy is superior to all other FSL-based HSI classification methods. SPN boosts OA by 1% for IP, 4.28% for Salinas and 2% for Houston over prototypical networks at $n_{\text{times}} = 25$. We observed that with higher K-shot and n_{times}, intersample repetitive features offset the prototype.

Authorized licensed use limited to: Shah and Anchor Kutchhi Engineering College. Downloaded on July 20,2022 at 08:18:40 UTC from IEEE Xplore. Restrictions apply.

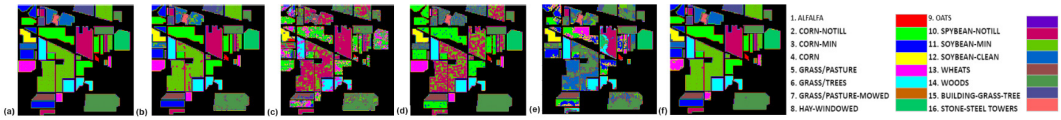

Fig. 4. Five-shot classification result of IP dataset. (a) Ground truth. (b) Prototypical networks. (c) Relation networks. (d) Matching network. (e) MAML. (f) SPN.

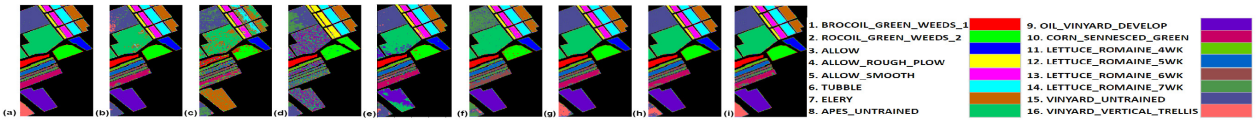

Fig. 5. Classification result of Salinas dataset. (a) Ground truth. (b) Prototypical networks-5 shot. (c) Relation networks-5 shot. (d) Matching network-5 shot. (e) MAML-5 shot. (f) SPN-1 shot. (g) SPN-5 shot. (h) SPN-10 shot. (i) SPN-15 shot.

IP classification map in Fig. 4 shows that only SPN could detect "Soybean-Min" class samples accurately. Prototypical networks performed similar to SPN, but it could not correctly classify some samples of this class. For Salinas, SPN detects classes having fewer samples, as shown in Fig. 5. When SPN is fine-tuned with a higher number of support images, the predicted classification map tends to be similar to ground-truth map.

IV. Conclusion

We proposed a novel method to gain model confidence in FSL. We incorporated DropBlock in our encoder and minimized prediction variance by passing the samples multiple times through the encoder to make the model learn robust representations for effective classification. By minimizing variance loss along with CE loss, SPN makes the model confident in predicting classes having very limited spectral–spatial samples. The proposed method can be incorporated in any few-shot-based method to gain stability. SPN fully utilizes the parameter space to build stable class prototypes and outperforms the current state-of-the-art (SOTA) FSL methods for HSI classification.

References

[1] B. Tu, C. Zhou, X. Liao, G. Zhang, and Y. Peng, "Spectral–spatial hyperspectral classification via structural-kernel collaborative representation," *IEEE Geosci. Remote Sens. Lett.*, vol. 18, no. 5, pp. 861–865, May 2021.

[2] A. Sellami, A. B. Abbes, V. Barra, and I. R. Farah, "Fused 3-D spectral–spatial deep neural networks and spectral clustering for hyperspectral image classification," *Pattern Recognit. Lett.*, vol. 138, pp. 594–600, Oct. 2020.

[3] Y. Chen, Z. Lin, X. Zhao, G. Wang, and Y. Gu, "Deep learning-based classification of hyperspectral data," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 7, no. 6, pp. 2094–2107, Jun. 2014.

[4] C.-I. Chang, Q. Du, T.-L. Sun, and M. L. G. Althouse, "A joint band prioritization and band-decorrelation approach to band selection for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 37, no. 6, pp. 2631–2641, Nov. 1999.

[5] Y. Fang *et al.*, "Dimensionality reduction of hyperspectral images based on robust spatial information using locally linear embedding," *IEEE Geosci. Remote Sens. Lett.*, vol. 11, no. 10, pp. 1712–1716, Oct. 2014.

[6] C. Rodarmel and J. Shan, "Principal component analysis for hyperspectral image classification," *Surv. Land Inf. Syst.*, vol. 62, no. 2, pp. 115–122, Jun. 2002.

[7] W. Li and Q. Du, "Gabor-filtering-based nearest regularized subspace for hyperspectral image classification," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 7, no. 4, pp. 1012–1022, Apr. 2014.

[8] P. Ghamisi, M. D. Mura, and J. A. Benediktsson, "A survey on spectral–spatial classification techniques based on attribute profiles," *IEEE Trans. Geosci. Remote Sens.*, vol. 53, no. 5, pp. 2335–2353, May 2015.

[9] R. Hang, Z. Li, Q. Liu, P. Ghamisi, and S. S. Bhattacharyya, "Hyperspectral image classification with attention-aided CNNs," *IEEE Trans. Geosci. Remote Sens.*, vol. 59, no. 3, pp. 2281–2293, Mar. 2021.

[10] R. Hang, Z. Li, P. Ghamisi, D. Hong, G. Xia, and Q. Liu, "Classification of hyperspectral and LiDAR data using coupled CNNs," *IEEE Trans. Geosci. Remote Sens.*, vol. 58, no. 7, pp. 4939–4950, Jul. 2020.

[11] B. Rasti *et al.*, "Feature extraction for hyperspectral imagery: The evolution from shallow to deep: Overview and toolbox," *IEEE Geosci. Remote Sens. Mag.*, vol. 8, no. 4, pp. 60–88, Dec. 2020.

[12] M. E. Paoletti *et al.*, "Capsule networks for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 4, pp. 2145–2160, Apr. 2019.

[13] W. Li, C. Chen, M. Zhang, H. Li, and Q. Du, "Data augmentation for hyperspectral image classification with deep CNN," *IEEE Geosci. Remote Sens. Lett.*, vol. 16, no. 4, pp. 593–597, Apr. 2019.

[14] Y. Chen, X. Zhao, and X. Jia, "Spectral–spatial classification of hyperspectral data based on deep belief network," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 8, no. 6, pp. 2381–2392, Jun. 2015.

[15] G. Koch, R. Zemel, and R. Salakhutdinov, "Siamese neural networks for one-shot image recognition," in *Proc. ICML Deep Learn. Workshop*, Lille, France, vol. 2, 2015. [Online]. Available: https://scholar.google.com/scholar?hl=en&as_sdt=0%2C5&q=Siamese+neural+networks+for+one-shot+image+recognition&btnG=

[16] C. Finn, P. Abbeel, and S. Levine, "Model-agnostic meta-learning for fast adaptation of deep networks," 2017, *arXiv:1703.03400*. [Online]. Available: http://arxiv.org/abs/1703.03400

[17] J. Snell, K. Swersky, and R. Zemel, "Prototypical networks for few-shot learning," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, pp. 4077–4087.

[18] O. Vinyals *et al.*, "Matching networks for one shot learning," in *Proc. Adv. Neural Inf. Process. Syst.*, 2016, pp. 3630–3638.

[19] B. Liu, X. Yu, A. Yu, P. Zhang, G. Wan, and R. Wang, "Deep few-shot learning for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 4, pp. 2290–2304, Apr. 2019.

[20] K. Gao, B. Liu, X. Yu, J. Qin, P. Zhang, and X. Tan, "Deep relation network for hyperspectral image few-shot classification," *Remote Sens.*, vol. 12, no. 6, p. 923, Mar. 2020.

[21] F. Sung, Y. Yang, L. Zhang, T. Xiang, P. H. S. Torr, and T. M. Hospedales, "Learning to compare: Relation network for few-shot learning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2018, pp. 1199–1208.

[22] S. K. Roy, G. Krishna, S. R. Dubey, and B. B. Chaudhuri, "HybridSN: Exploring 3-D–2-D CNN feature hierarchy for hyperspectral image classification," *IEEE Geosci. Remote Sens. Lett.*, vol. 17, no. 2, pp. 277–281, Feb. 2020.

[23] G. Ghiasi, T.-Y. Lin, and Q. V. Le, "DropBlock: A regularization method for convolutional networks," in *Proc. Adv. Neural Inf. Process. Syst.*, 2018, pp. 10727–10737.

[24] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," *J. Mach. Learn. Res.*, vol. 15, no. 1, pp. 1929–1958, 2014.

[25] J. Tompson, R. Goroshin, A. Jain, Y. LeCun, and C. Bregler, "Efficient object localization using convolutional networks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, Jun. 2015, pp. 648–656.

[26] A. Banerjee, S. Merugu, I. S. Dhillon, and J. Ghosh, "Clustering with Bregman divergences," *J. Mach. Learn. Res.*, vol. 6, no. 4, pp. 1705–1749, 2005.

[27] Y. Gal and Z. Ghahramani, "Dropout as a Bayesian approximation: Representing model uncertainty in deep learning," in *Proc. Int. Conf. Mach. Learn.*, 2016, pp. 1050–1059.

[28] A. Damianou and N. Lawrence, "Deep Gaussian processes," in *Proc. Artif. Intell. Statist.*, 2013, pp. 207–215.

Authorized licensed use limited to: Shah and Anchor Kutchhi Engineering College. Downloaded on July 20,2022 at 08:18:40 UTC from IEEE Xplore. Restrictions apply.